Werkproces uitwerking SD-Portfolio

Werkproces	K1W2: Maakt een technisch ontwerp voor software
Studentnaam	Yakub Bulut
Studentnummer	97118569
Klascode	SDA3
Datum oplevering	14/09/2026

Student verklaart hierbij dat de bewijsstukken in dit STARR-document

- Authentiek zijn
- Kwalitatief goed zijn
- Binnen een redelijke tijdsduur tot stand zijn gekomen

Bewijsstuk

Per bewijsstuk moet er gekeken worden naar hoe sterk deze is, hiervoor kijk je naar twee dingen:

1. De context – Waar en met wie heb je het gedaan?
2. Je zelfstandigheid– Hoe zelfstandig heb je gewerkt?

Voor zowel de context als de zelfstandigheid kan een score A (hoog), B (middelmatig) of C (laag) gegeven worden. Een bewijsstuk met de score ‘A - A’ (goede context én zelfstandig gewerkt) heeft een hogere waarde dan een bewijsstuk met de score ‘B - C’.

Maak voor het bepalen van de complexiteit gebruik van de "Context check" en de "Zelfstandigheid check”.

Korte beschrijving	Context	Zelfstandigheid
In deze bewijsstuk leg ik uit over hoe ik mijn request_repair heb gebouwd voor mijn eigen bedrijf.	Alleen Context check Uitkomst (B)	Alleen Zelfstandigheid check uitkomst (B)

STARR-reflectie

Situatie	Voor mijn onderneming YB Repairs werd ik vooral via Instagram of vrienden benaderd voor computerreparaties en pc's op maat. Ik had nog geen werkende website waarop klanten zelf een aanvraag konden indienen. Ik was eerder al bezig met een website, maar die had nog geen functionaliteiten. Sinds het begin van het nieuwe schooljaar werk ik daarom aan een nieuwe website in Laravel. Hierop kunnen klanten een account aanmaken, aanvragen indienen en hun status bekijken. Als beheerder kan ik aanvragen accepteren, weigeren en archiveren. Voor het bewijsstuk 'Ontwerpen' werk ik de gebruikerswensen, schermen en database uit. Zo laat ik zien hoe de website is opgebouwd en waarom ik bepaalde ontwerpkeuzes heb gemaakt. **15-9-2026**
Taak	• **Databaseontwerp maken:** een ERD uitwerken met de tabellen users en repair_requests. Daarbij laat je zien welke gegevens worden opgeslagen en hoe één gebruiker meerdere reparatieaanvragen kan hebben. • **Ontwerpkeuzes uitleggen:** beschrijven waarom je deze structuur gebruikt. Bijvoorbeeld waarom elke aanvraag een user_id heeft, waarom je een status opslaat en waarom je archived_at gebruikt om aanvragen naar de geschiedenis te verplaatsen.
Activiteiten	**1. Bepalen welke gegevens nodig waren** Ik heb eerst gekeken welke informatie ik nodig heb om een reparatieaanvraag te beoordelen. Een klant moet het apparaattype, merk, eventueel het model en een omschrijving van het probleem kunnen invullen. Daarnaast moest iedere aanvraag gekoppeld worden aan de klant die deze indient. **2. De databasetabel opbouwen** Daarna heb ik een Laravel-migratie gemaakt voor de tabel repair_requests. Hierin heb ik de kolommen en gegevenstypen vastgelegd. Voor de probleemomschrijving heb ik bijvoorbeeld text gebruikt, omdat deze langer kan zijn. Het model van het apparaat heb ik optioneel gemaakt met nullable(), omdat een klant dit niet altijd weet. **3. De aanvraag aan een gebruiker koppelen** Ik heb de kolom user_id toegevoegd als foreign key naar de tabel users. Hiermee is duidelijk van wie een aanvraag is. Vervolgens heb ik de relaties in de Laravel-modellen ingesteld: een gebruiker kan meerdere reparatieaanvragen hebben en iedere aanvraag hoort bij één gebruiker. **4. Status en geschiedenis uitwerken** Om aanvragen te kunnen beoordelen, heb ik het veld status gebruikt met de waarden in_afwachting, geaccepteerd en geweigerd. Later heb ik archived_at toegevoegd. Daarmee kan een aanvraag uit het actieve overzicht verdwijnen, terwijl de gegevens beschikbaar blijven in de geschiedenis. Archiveren en definitief verwijderen zijn daardoor twee verschillende acties. **5. De werking controleren en verbeteren**

	Ik heb gecontroleerd of een aanvraag werd opgeslagen en bij de juiste klant en de admin zichtbaar was. Daarna heb ik het accepteren, weigeren en archiveren gecontroleerd. Ook heb ik automatische tests uitgevoerd. Bij foutmeldingen heb ik de melding vergeleken met de controller en de bijbehorende pagina. Zo heb ik ontbrekende gegevens voor het dashboard aangevuld en daarna opnieuw getest. Tijdens het bouwen gebruikte ik uitleg en codevoorbeelden als ondersteuning. Ik bekeek hoe deze aansloten op mijn bestaande code en controleerde de werking in mijn eigen applicatie.
Resultaat	**Ontwerp*** **"De eisen en wensen zijn vertaald naar een passend, eenduidig en volledig ontwerp."** <uitwerking onderbouwing bewijs> Ik heb de wens om reparatieaanvragen te kunnen indienen vertaald naar de tabel repair_requests. In afbeelding 1 is te zien welke gegevens ik hiervoor heb vastgelegd. Iedere aanvraag krijgt een uniek nummer via id. Met user_id wordt de aanvraag gekoppeld aan de juiste klant. De velden device_type, brand, model en problem_description bevatten de informatie die ik nodig heb om het probleem te beoordelen. Het model is optioneel omdat een klant dit niet altijd weet. Daarnaast heb ik een statusveld toegevoegd om de voortgang bij te houden. Met timestamps() wordt opgeslagen wanneer een aanvraag is aangemaakt en voor het laatst is gewijzigd. Zo sluit de databasestructuur aan op het indienen en beheren van reparatieaanvragen.

2026_09_04_180830_create_repair_requests_table.php | RepairRequest.php | RepairRequestController.php 4 | 2026_09_11_120031_create_pc_build

mijn-project > database > migrations > 2026_09_04_180830_create_repair_requests_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('repair_requests', function (Blueprint $table) {
            $table->id();

            $table->foreignId('user_id')
                ->constrained()
                ->onDelete('cascade');

            $table->string('device_type');
            $table->string('brand');
            $table->string('model')->nullable();

            $table->text('problem_description');

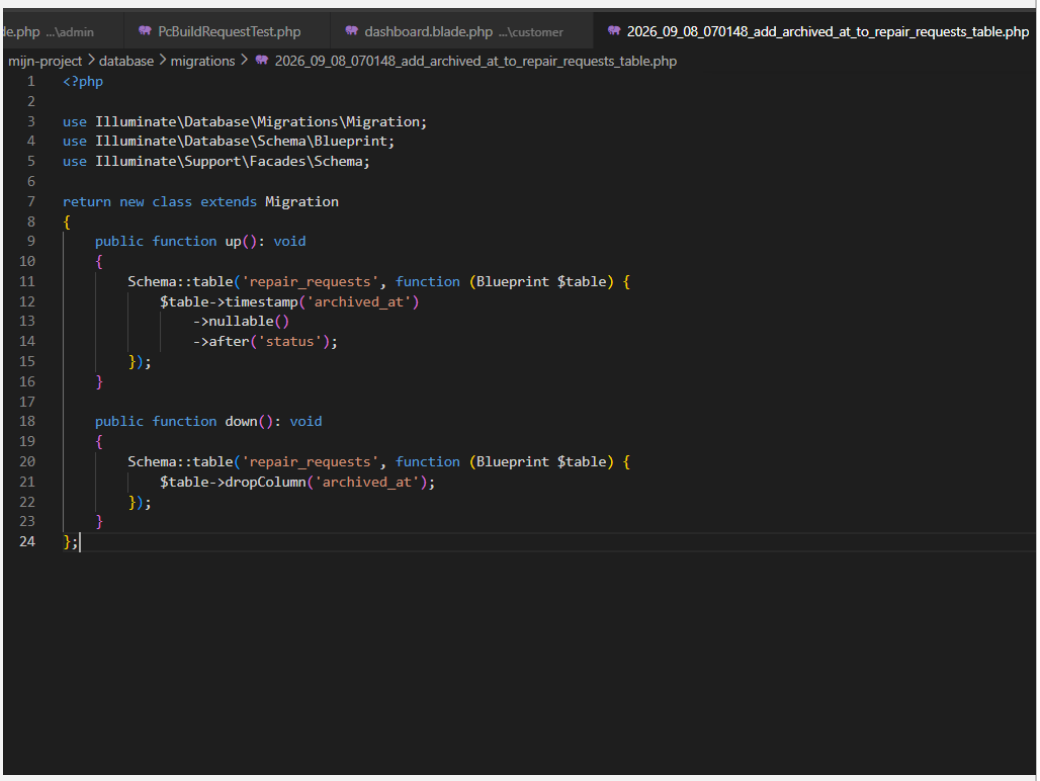
            $table->string('status')->default('ontvangen');

            $table->timestamps();
        });
    }

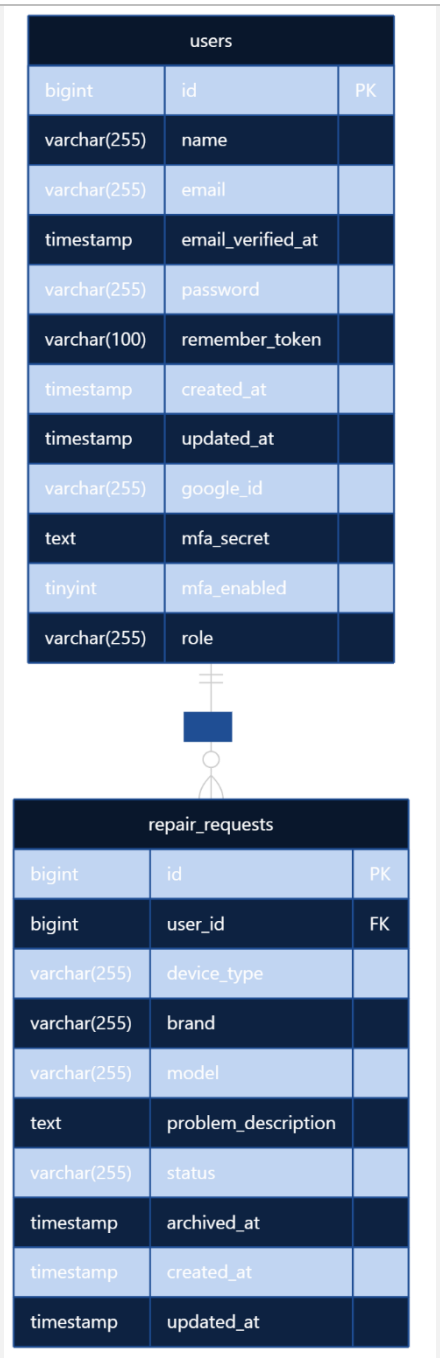
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('repair_requests');
    }
};

```

Ik heb de wens om reparatieaanvragen te kunnen indienen vertaald naar de tabel repair_requests. In afbeelding 1 is te zien welke gegevens ik hiervoor heb vastgelegd. Iedere aanvraag krijgt een uniek nummer via id. Met user_id wordt de aanvraag gekoppeld aan de juiste klant. De velden device_type, brand, model en problem_description bevatten de informatie die ik nodig heb om het probleem te beoordelen. Het model is optioneel, omdat een klant dit niet altijd weet.
Daarnaast heb ik een statusveld toegevoegd om de voortgang bij te houden. Met timestamps() wordt opgeslagen wanneer een aanvraag is aangemaakt en voor het laatst is gewijzigd. Zo sluit de databasestructuur aan op het indienen en beheren van reparatieaanvragen.



	Voor het databaseontwerp heb ik een ERD uitgewerkt op basis van de bestaande tabellen users en repair_requests. In afbeelding 3 staan de kolommen en de verbinding tussen deze tabellen. Iedere gebruiker heeft een uniek id. De kolom user_id in repair_requests verwijst naar dit nummer. Hierdoor is vastgelegd bij welke klant een aanvraag hoort. Eén gebruiker kan meerdere reparatieaanvragen hebben, maar iedere aanvraag hoort bij precies één gebruiker. In het schema staat PK voor de primaire sleutel, het unieke nummer van een record. FK staat voor foreign key de sleutel die de tabellen met elkaar verbindt. Het ERD maakt deze opbouw overzichtelijk en helpt mij te controleren of de database aansluit op de werking van de reparatieaanvragen. *Afbeelding 3: ERD van gebruikers en reparatieaanvragen.*



Haalbaarheid en keuzes
Ik gebruik Laravel-migraties en modelrelaties, omdat deze passen bij mijn project. Klantgegevens staan in users en reparatiegegevens in repair_requests. Door de koppeling met user_id hoef ik klantgegevens niet bij iedere aanvraag opnieuw op te slaan.

Het apparaatmodel is optioneel, omdat een klant dit niet altijd weet. Met het statusveld houd ik bij of een aanvraag in afwachting, geaccepteerd of geweigerd is. Met archived_at kan ik een aanvraag bewaren in de geschiedenis zonder deze te verwijderen.

Privacy
Een klant mag alleen zijn eigen aanvragen zien. Daarom haalt de controller

	alleen aanvragen op die bij het ingelogde account horen. Zo verschijnen aanvragen van andere klanten niet in het overzicht. **Beveiliging** Alleen een admin mag aanvragen accepteren, weigeren en verwijderen. De routes controleren daarom of iemand is ingelogd en de rol admin heeft. Ook controleer ik de ingevulde gegevens, zoals verplichte velden en de maximale tekstlengte. Het gebruikersnummer komt automatisch uit het ingelogde account.
Reflectie	• **Kwaliteit van het eindproduct** Ik ben tevreden met het ontwerp. Het ERD laat duidelijk zien hoe gebruikers en reparatieaanvragen gekoppeld zijn. De gebruikte status namen wil ik nog overal gelijkmaken. • **Benodigde tijd** Ik heb ongeveer 3 uurtjes besteed aan de screenshots, het ERD en de uitleg van mijn keuzes. • **Verloop van het proces** Tijdens het uitwerken begreep ik de database en de relaties steeds beter. De volgende keer maak ik het ERD vóór het programmeren, zodat de structuur vooraf duidelijk is. • **Samenwerking en overleg** Ik heb vooral zelfstandig gewerkt en uitleg gevraagd aan docenten wanneer iets onduidelijk was. De volgende keer wil ik mijn ontwerp ook door een docent of medestudent laten controleren.

https://drive.google.com/file/d/1XYGi2LsspJaa8lp2dkyxoBb1XZnJ9vnI/view?usp=drive_link

EEN LINK NAAR DE DEMO!!!!!!!!!!